AI Usage Log

retail-loan-approval-poc *Demo | Tools, human decisions, corrections and validation*

Tools used

Claude (claude.ai, including web search and code-running tools) and ChatGPT (chatgpt.com). I used them as assistants for implementation, research, troubleshooting and documentation. I reviewed and tested the resulting work.

My role and decisions

- Set priorities from the assignment brief: build the demo first, then complete the research note, policy agent, architecture pages and submission materials.
- Chose not to conduct practitioner interviews because access and time were unavailable; the research note states this limitation explicitly.
- Created and configured the Google Cloud project and BigQuery dataset, installed the Cloud CLI, authenticated, configured environment variables and loaded the data.
- Diagnosed local setup issues, including a stale module import and missing Google credentials, and reran the application.
- Tested the approve, accept and disburse journey and checked that the application identified BigQuery as its data source.
- Decided to describe the agents accurately as deterministic, rule-based components rather than claiming they use an LLM. I also edited the code and documents to fit my own presentation style.

How AI assistance was used

AI-assisted work

Streamlit page structure, synthetic applicant data, EMI and FOIR calculations, SQLite persistence, BigQuery loading with local fallback, and policy-clause retrieval.

Research: Searches for RBI KYC, digital lending, Key Facts Statement and DPDP sources; drafting and organising the research note with regulations, practice considerations and demo assumptions distinguished.

Documentation: Drafting the approach narrative, business and technical architecture, About and FAQ pages, and README.

Testing Suggestions for unit tests and headless page checks; help investigating errors and correcting implementation issues.

Representative prompts

- “Use this particular thing from theassignment brief to help build a working Streamlit demo.”
- “What have I done, and what remains? Help me fix the gaps.”
- “Draft the research note; I cannot conduct practitioner interviews.”
- “How do I configure BigQuery?”; “Policy agent next”; “Create the business architecture view.”
- “Can I accurately describe this as agentic AI if I have not used an LLM?”

What I accepted, changed or rejected

- Accepted the overall application structure, amortisation formula, layered architecture, research-note structure and human-in-the-loop decision design.
- Expanded the initial build after reviewing it against the brief; added data-layer functionality, workflow stages, policy retrieval and architecture content.
- Replaced a deprecated Streamlit argument. A unit test exposed an incorrect policy-clause match for loan size, which was corrected; the architecture status was also updated after BigQuery integration.
- Kept the credit-reporting frequency claim unconfirmed because the sources reviewed did not agree on the start date.
- Avoided describing the rule-based components as LLM-powered agents. The demo uses synthetic data and mocked checks; it is not a production lending system.

Validation performed

Reported checks: 7 unit tests covering EMI, FOIR, synthetic data generation, SQLite round-trip, policy retrieval and rules; a headless load check for all four pages; recomputation of README scenario outcomes against the code; and a manual approve → accept → disburse run. I also checked that the sidebar displayed BigQuery as the active data source.

Transparency note: AI tools supported drafting and coding, but I made the scope and disclosure decisions and performed the listed checks. Test counts and outcomes should be retained only if they match the final submitted version.